

Data Collection Protocol v4.4

Changelog v4.1 to v4.2

- Added “Calibration Extraction” — the walk-about bag must sit inside a calibration parent folder, since extraction expects a multi-episode session
- Added “Metadata.json” — all fields documented; `exo_calib_notes` is informational only and never fails the bag
- Removed the `rebuild.sh` step from “Exo-Camera Calibration ROS Node component” — runs `exo_calib_record.sh` directly
- Removed the `rebuild.sh` step from “Run Recorder” — runs `record.sh` directly

Changelog v4.2 to v4.3

- Added more details to the 'list of equipment' slide
- List of scripts slide → list of scripts in leo partners repository
- 'Extraction Scripts' slide update to use compressed ego colour topic
- 'Episodes' slide emphasise that the episodes should be based on staff's own SOP.

Changelog 4.3 to 4.4

- **Protocol changes**
- 'Session' and 'Episodes' slides consolidate the key definitions for session, episodes and transitions — session is defined first, then episodes, then transitions
- 'Episodes' slide adds the transition rule — navigation $\geq 5s$ becomes its own transition episode, coded <preceding>-<succeeding>
- 'Mistakes' slide new mistake added m3 — forgets which step / attempted wrong step
- 'Calibration' slide states when recalibration is required — bed position, camera rotation or position, or any change that cannot be returned to the calibrated state
- 'Recording Overview' slide defines a session as 1 workflow pass $\times$ 1 subject $\times$ 1 bed type

Changelog 4.3 to 4.4

- **Scripts and outputs**
- 'Metadata.json' slide is updated with the new metadata format — adds expected_streams and step_errors; missing and extra stream errors are now written by the validator diff
- 'Exo-Camera Calibration — Solve' tightens the fx_spread_pct target to $<\sim 3\%$
- 'Calibration' slide: calibration JSON is kept in the session folder for housekeeping; extraction writes it into metadata.json

Changelog 4.3 to 4.4

- **Slides added, merged and renamed**
- Added 'Example of the Episode List Part (3)' — shows a transition episode row (a-b)
- Removed the two duplicate 'Overview' slides at the front; merged 'Rosbag files' into 'Recording output files'
- Camera, calibration and fault slides retitled by stage — Corner Placement / Heights & Mounts / Facing Direction, Capture / Solve, Startup / During Recording

Overview

1 Data to be Collected

Episodes, actions and objects — how each subtask is described and coded.

2 Consent & privacy

IRB agreement and participant sign-off before any recording begins.

3 Set up the rig

Cameras, firmware, depth calibration and workspace build. Once per machine.

4 Record a session

One subject, one workflow pass; naturalistic mistakes are kept and tagged.

5 Stop & verify

Check each bag — duration, topics and frame rate — before moving on.

6 Extract

Wrapper script turns each session's bags into one folder per episode.

7 Submit

Output directory holds every session's files, ready to share.

List of equipment

- RealSense D435i/if x1 – Ego
 - Colour + Depth + IMU
 - 1280 × 720 @ 30 fps
 - Fixed focus
 - Requires USB 3.1 Gen1 (5 Gbps)
- Logitech C922 Pro Stream Webcam x4 – Exo
 - Colour only
 - 1280 × 720 @ 30 fps
 - MJPEG over USB 2.0
 - Fixed focus, fixed exposure
 - 480 Mbps — expected, not a fault
- Active USB extension cables for cameras.
 - Active USB 2.0 for Logitech C922 Pro x4 (5-7m)
 - Active USB 3.0 (5Gbps) Realsense D435i x1 (5m)



List of equipment

- Camera stands (2m height) x4 – exo cameras
 - With standard camera mount with a ball joint to allow rotation of the camera.
 - Standard ¼ inch screw mount
- Action Camera headband with GoPro Mount + GoPro to standard ¼ inch screw tripod mount
 - The Realsense camera uses the standard ¼ inch mount
- For wire management, any chest camera rig with shoulder straps and waist straps, with velcro.
 - This is merely to reduce wire tangle and stress on the USB ports.



List of scripts in leo partners

- Data Collection ROS Node
 - In folder 'Leo-data-collection'
- Exo Camera Calibration
 - In folder 'Leo-exo-calib'
- Data Extraction
 - In folder 'data-processing-scripts-v3'



Documenting the bedmaking workflow

Session

- Each hotel has its own bedmaking process.
 - We want to follow the staff's process. We do not want to invent one for them to follow.
- We use the term session to describe one complete workflow pass
- The session can be broken down into discrete subtasks called episodes:
 - The process is recorded as a sequential list of episodes
 - Each episode must therefore be recorded in workflow order
 - The episode sequence is captured by each recording's start time.
- One session's episodes example:
 1. Take pillow and put it on bed,
 2. Take pillowcase and unfold it on the bed
 3. Pick up pillow and insert into pillowcase.
 4. Put fitted pillow at the head of bed, leaning against headboard at a 45-degree angle

Episodes

- Each episode is a discrete subtask of the workflow
 - Each episode is given a unique ID Code
 - To ensure unique codes, we suggest adding the hotel's name/shorthand in front of the code.
 - e.g. SUTD1, MyHotel24
- Transition episodes
 - These episodes do not encode a subtask.
 - They encode navigation between subtasks.
 - If the duration to move from one subtask to another $\geq 5s$, then it can be its own transition episodes.
 - E.g. In step 4, a person moves with the pillow to the headboard to place it. This counts as part of the episode, not a transition.
 - E.g. After completing step 2, the subject must walk, without interacting with any object, to a new position before starting step 3.
 - This movement takes 5s and hence it is a transition episode with code 2-3, as it encodes the activity occurring between episodes 2 and 3
 - A movement duration $\geq 5s$, means the navigation is significant enough to be encoded on its own and not as part of a subtask

Episodes

1. Take pillow and put it on bed,
2. Take pillowcase and unfold it on the bed
3. Pick up pillow and insert into pillowcase.
4. Put fitted pillow at the head of bed, leaning against headboard at a 45-degree angle

Episode's Actions, Objects

- An episode contains
 - Objects: The list of objects should focus on the main object involved. (e.g. when tucking in the duvet under the mattress, focus on the duvet, not the mattress)
 - Actions: The word should describe the action taken. Use multiple words if several actions occur. (e.g. wave, tuck)
 - Step Description: describes the action in a short sentence
 - Episode code: uniquely identifies the episode (e.g SUTD1, 1)

Example of the Episode List Part (1)

Codes are IDs not
execution order

Code	Step description	Actions	Objects
1	Remove pillow case from pillow	['pull', 'lift']	['pillow','pillowcase']
43	put aside pillowcover	['carry']	['pillowcase']
44	Move pillow aside	['carry']	['pillow']
45	Untuck duvet (if tucked)	['pull','lift']	['duvet']
3	Remove duvet from duvet cover, leave on mattress	['pull', 'lift']	['duvet','duvet cover']
35	Fold duvet and place on mattress	['fold','lift']	['duvet']
4	Move duvet aside	['pull', 'lift']	['duvet']
36	Fold duvet cover and place on mattress	['fold','lift']	['duvet cover']
5	Move duvet cover aside	['carry']	['duvet cover']
6	Remove bedsheet from mattress, leave on mattress	['pull', 'lift']	['bedsheet']
37	Fold bedsheet and place on mattress	['fold','lift']	['bedsheet']
7	Move bedsheet aside	['carry']	['bedsheet']
8	Remove mattress protector from mattress, leave on mattress	['pull', 'lift']	['mattress protector']
38	Fold mattress protector and place on mattress	['fold','lift']	['mattress protector']
9	Move mattress protector aside	['carry']	['mattress protector']

Example of the Episode List Part (2)

Codes are IDs not
execution order

Code	Step description	Actions	Objects
10	place mattress protector on bed	['carry']	['mattress protector']
11	wave mattress protector	['shake','lift']	['mattress protector']
12	tuck in mattress protector	['fitting','tuck']	['mattress protector']
13	place bedsheet on the bed	['carry']	['bedsheet']
14	wave bedsheet	['shake','lift']	['bedsheet']
15	tuck in bedsheet	['fitting','tuck']	['bedsheet']
16	place duvet cover on bed	['carry']	['duvet cover']
29	Wave duvet cover	['shake','lift']	['duvet cover']
17	Place duvet on bed	['carry']	['duvet']
31	Wave duvet	['shake','lift']	['duvet']
18	Put duvet into duvet cover	['stuff','shake','lift']	['duvet','duvet cover']
19	Wave and align duvet on bed	['fitting','tuck','shake']	['duvet']
42	Tuck duvet	['fitting','tuck']	['duvet']

Example of the Episode List Part (3)

Code	Step description	Actions	Objects
63	fit pillowcases on the table	['stuff','shake']	['pillow','pillowcase']
64	place pillow on bed and align	['carry']	['pillow']
a-b	Walking (Transition)	['walking']	[]

Codes are IDs not
execution order

Recording Mistakes

- We allow naturalistic mistakes to be included in the recording.
- Mistakes that are not relevant:
 - Using handphone
 - Slowing down or stopping to interact with other members of staff.
 - being called away to do other duties
- The mistakes should be relevant to the bed making process, such as:
 - Dropping beddings on the floor
 - Low Quality – requires another action to fix or redo full episode
 - The fix can be done within the current episode take/recording if possible
 - If the take/recording has ended, label the first take as containing a mistake and record a second take.
 - The person forgets which steps they are recording and does something different.

Mistakes

Code	Step description
M1	Dropping beddings on the floor
M2	Low Quality – requires another action to fix or redo full episode
M3	Forget which step to do/attempted wrong step.

Subject actions

- Extracted data is to be mapped to robot simulations.
- Try to move slower to allow the cameras to capture movements clearly
- Actively keep manipulation within the ego camera's field of view
 - Subject may need to actively turn their head to point the ego camera at what their hands are manipulating.
 - When carrying beddings around, it is helpful to lift the object so it can be captured by the ego camera.
 - Keep your arm span small — less than 1m
 - When carrying items, use both hands.
- Do not lean more than 1m away from torso. Keep manipulations close to torso.
- Bending and squatting are ok.

Consent and Privacy

Consent and Privacy

- Data is collected according to SUTD's IRB Code
 - IRB-25-779. Each participant declares consent by signing this IRB's consent form.
 - The signed consent forms will be stored as a record of subject agreement
 - The form PDF will be distributed directly to partner's liaison
- Dataset
 - The data will be stored without any personally identifiable information; consent form data cannot be traced to any dataset entry.
 - Only a UUID is kept to track individual subjects across videos. This is to track population size.
 - Faces will be blurred before the dataset is published.

Consent and Privacy

- Agree to the IRB statements on pages 5–6
 - **Items 7–10:** circle "Agree" on each one
 - **Item 11:** tick any one box — taking part does not require being quoted by name
 - **Item 12:** optional — the participant may tick yes or no
 - **Item 13:** tick yes and give at least one contact detail only if item 12 was yes; otherwise tick no

7. I **allow/ do not allow*** my coded data (without personal identifiers) to be used in future research. I understand that all future related research will be subject to the approval of an Institutional Review Board.
8. I **agree / do not agree*** to the photo taking of my participation in the research.
9. I **agree / do not agree*** to the audio recording of my participation in the research.
10. I **agree / do not agree*** to the video recording of my participation in the research.
11. I give permission to be quoted in any subsequent publications resulting from this research; and for the following personal identifiers to be disclosed in the investigators' academic publications, if any:
☐ surname ☐ first name ☐ organization name ☐ my title
☐ disagree.
12. I **agree / do not agree*** to be re-contacted for subsequent interview, if necessary.

***Please delete as appropriate**

13. Consent to be contacted for future research

- ☐ Yes, I agree to be contacted for future research that I may be eligible for.
I agree to be contacted via:
- ☐ Phone: _____
- ☐ Mail: _____
- ☐ Email: _____
- ☐ Others: _____

☐ No, I do not agree to be contacted for future research.

Consent and Privacy

- Sign on page 6
 - **Participant:** name, signature, date
 - **Consent taker:** name, signature, date
 - **Translator (** lines):** only if consent was taken in a language other than English
 - Enter the language, translator name and date in the ** sentence
 - If consent taker and translator are the same person, repeat the same name and signature on the translator line
 - Consent taken in English: leave the whole ** section blank

** This research has been explained to me in _____ (state language), which I understand, by _____ (name of translator) on _____ (date).

Name and Signature (Participant)

Date

Name and Signature (Consent Taker)

Date

** Name and Signature (Translator)

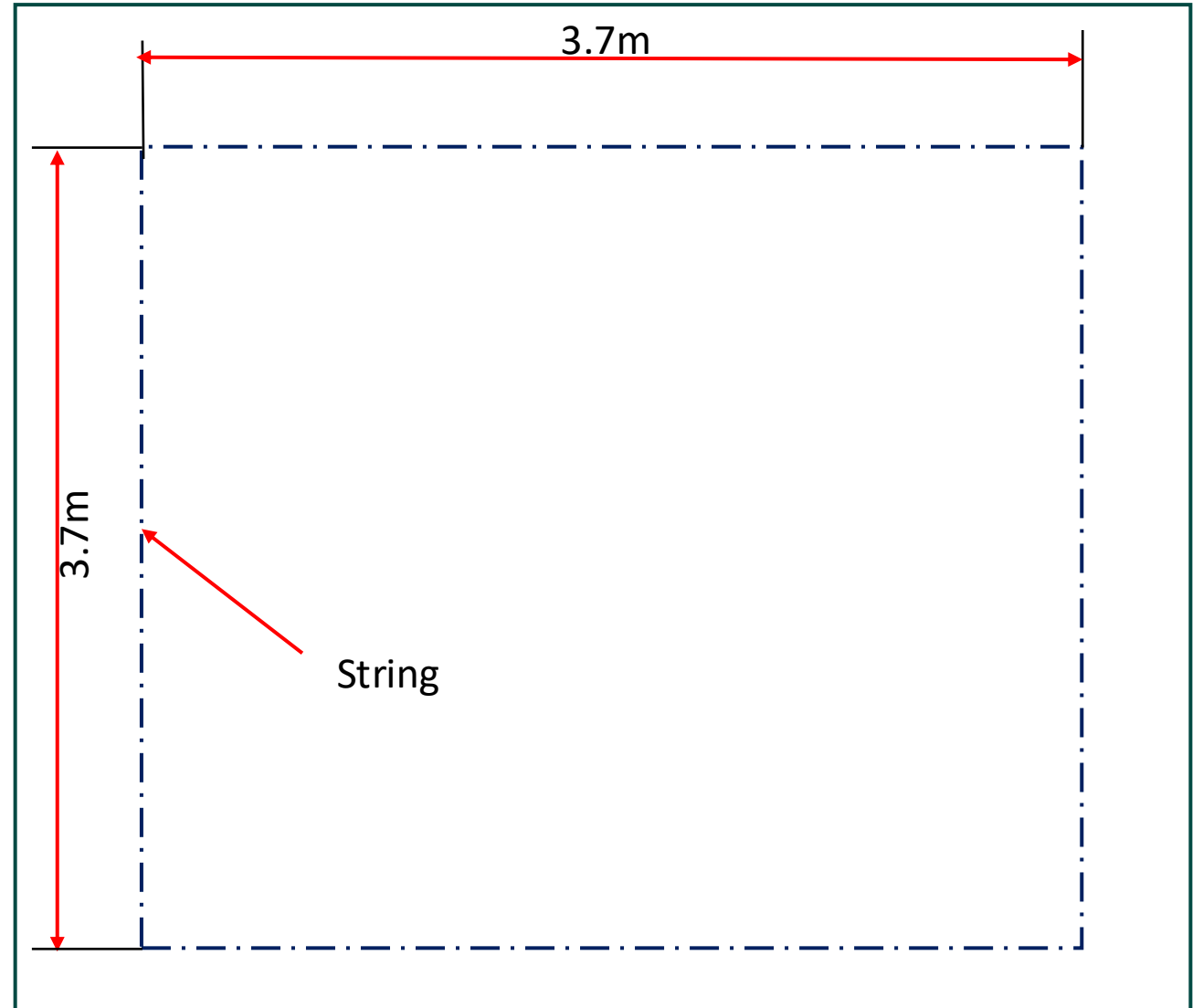
Date

*** (Please include this section if the participant is unable to understand English and read any of the translated consent documents available.)*

Set up the rig

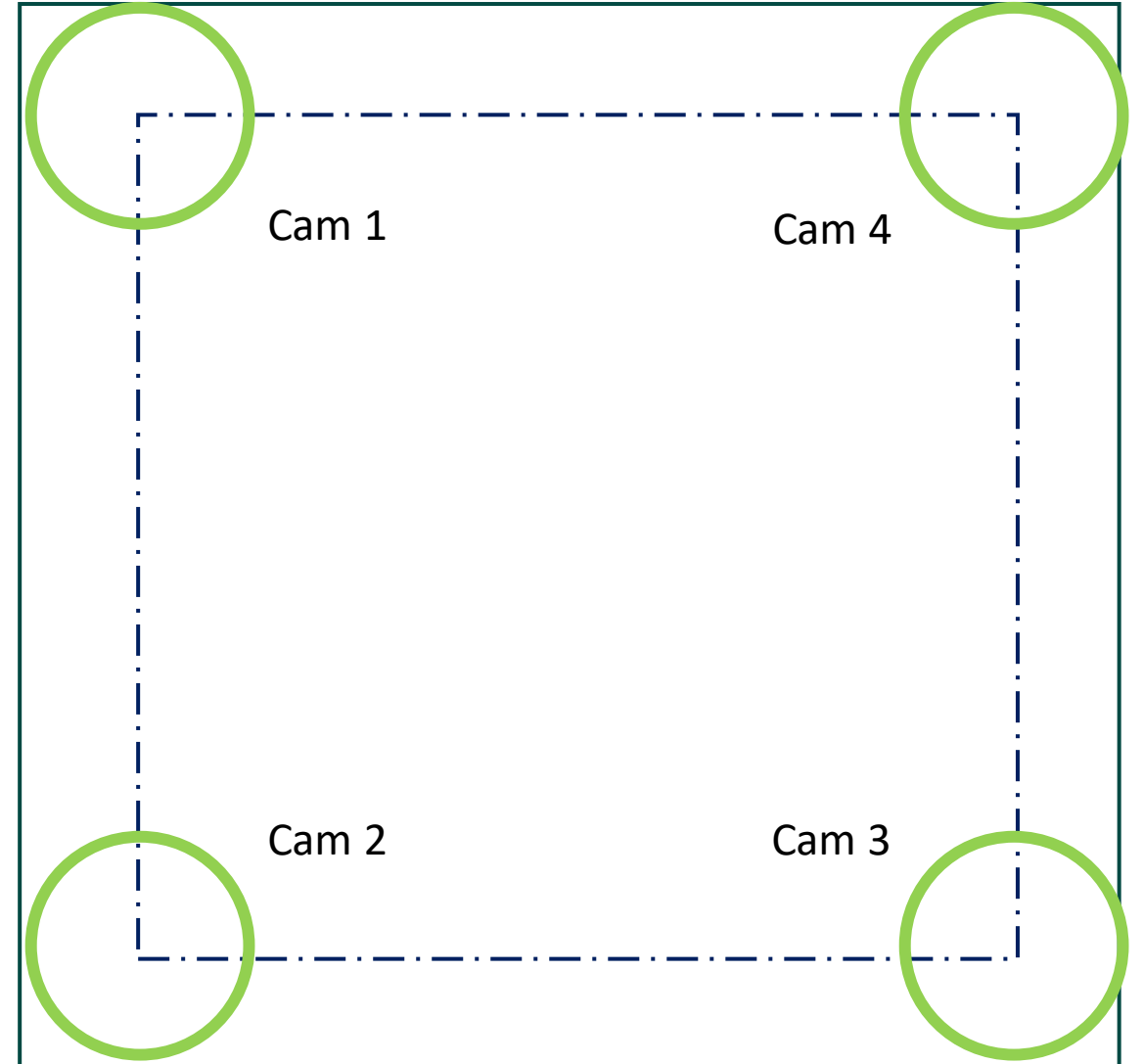
Base Setup

- Using a string and measuring tape, make a square (dashed lines)
- The size of the square differs according to the size of the bed. (E.g. we made a 3.7m square using string for a single bed.)
- The square helps us measure all the distances, which aids calibration of the exo camera setup.



Camera Stand Setup — Corner Placement

- The stands must be placed exactly at the corners of the square, as shown in the figure.



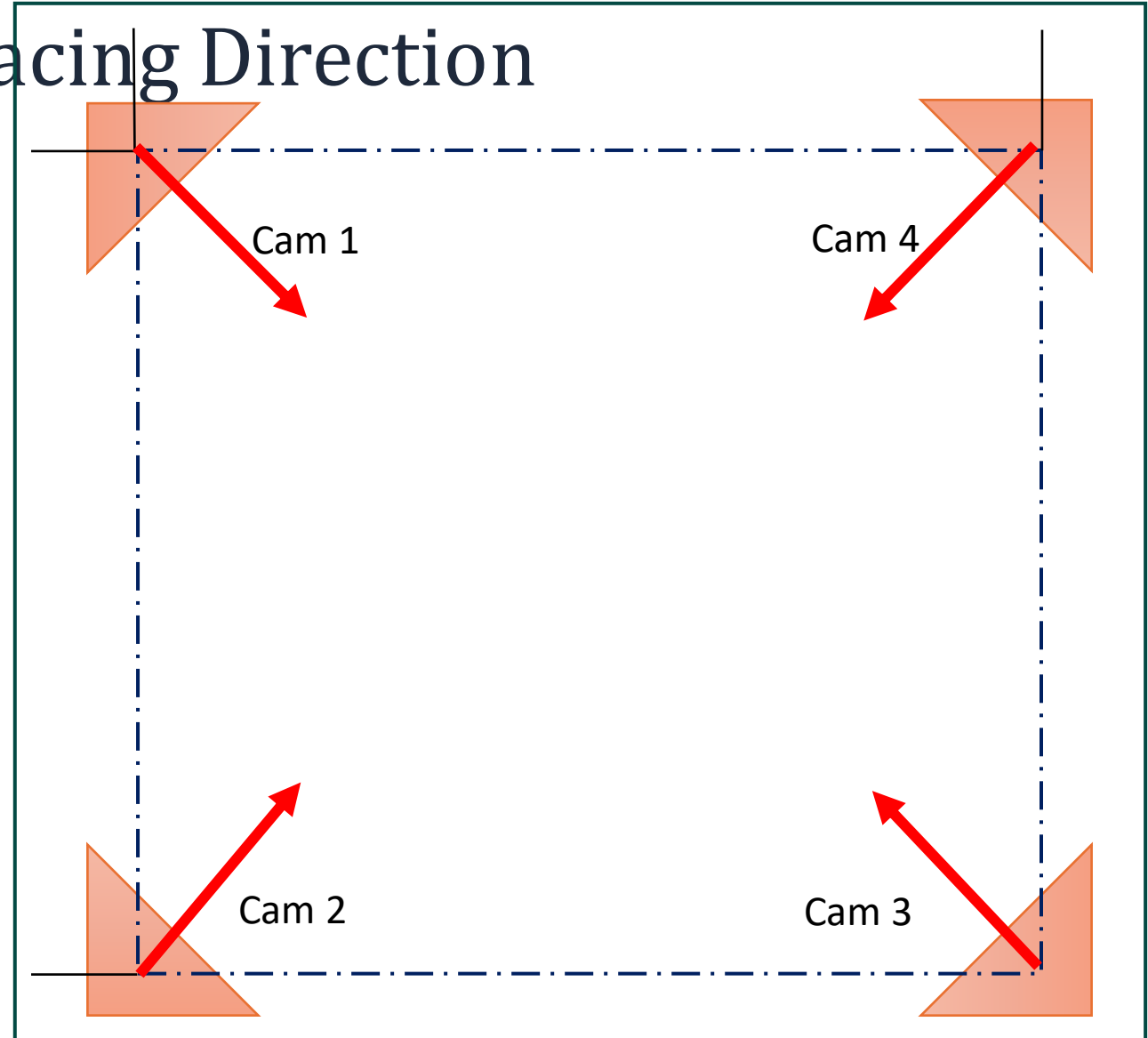
Camera Stand Setup — Heights & Mounts

- **Exo Cameras:** set up with camera stands to a height of 2m, measured from floor to base of camera.
- **Ego Camera:** mounted with an action-camera headband; use a harness for wire routing to reduce tangling.
- All cameras use a wired connection.



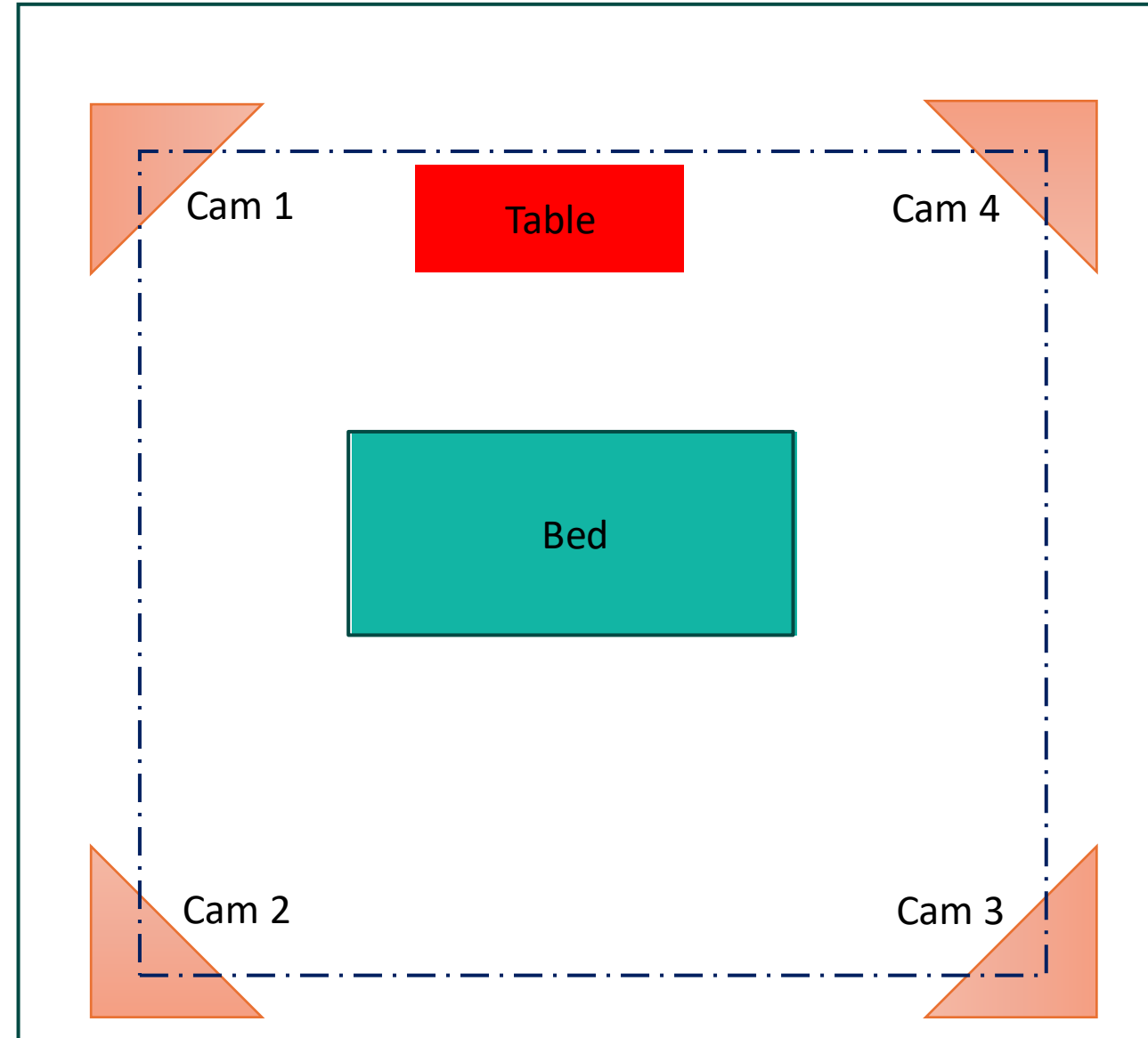
Camera Stand Setup — Facing Direction

- The cameras should be facing the centre of the square.
- Cam 1 and cam 3 face each other, as do cam 2 and cam 4.



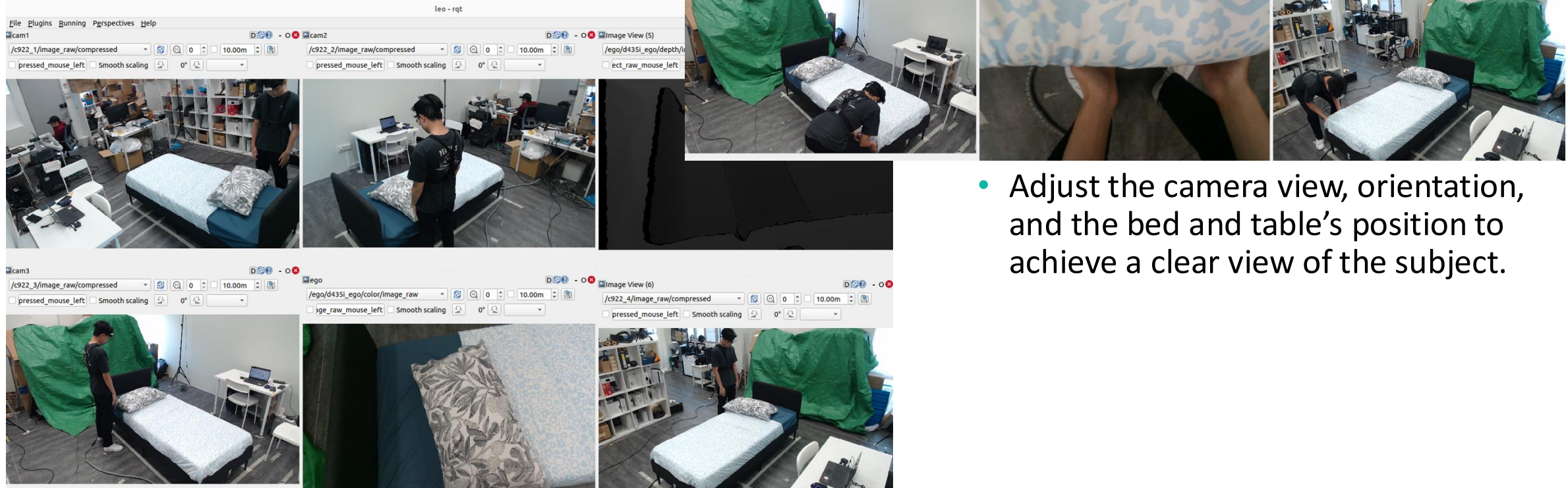
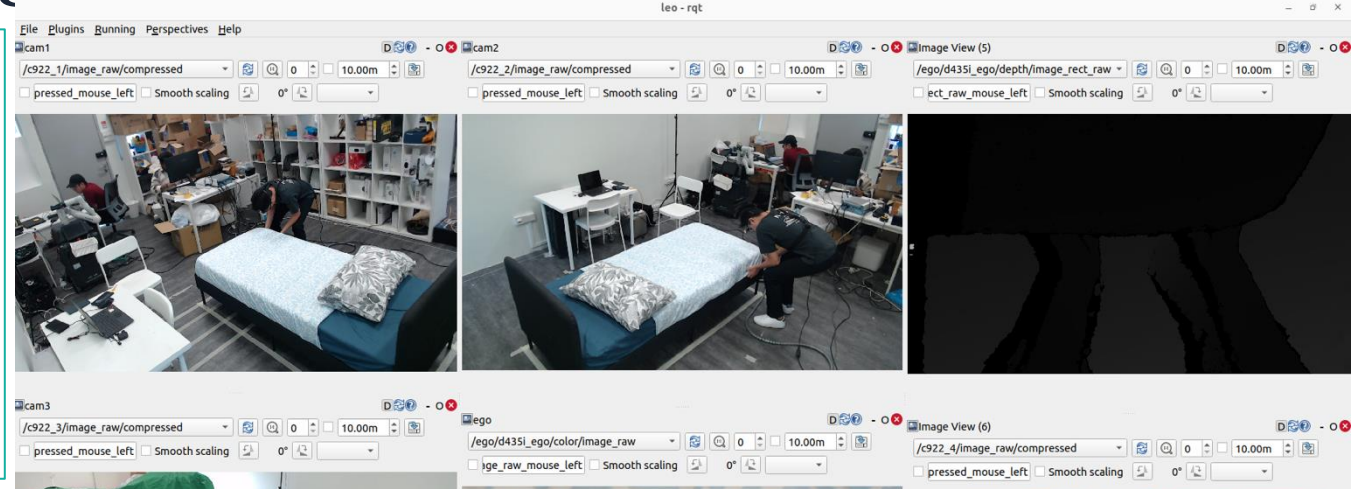
Bed and Table Setup

- The bed is placed roughly in the centre of the square, and the table is placed midway between cam 1 and cam 4.
- Ensure that the camera system is already set up and running. You may refer to the setup [here](#).



Camera Setup — Framing the Subject

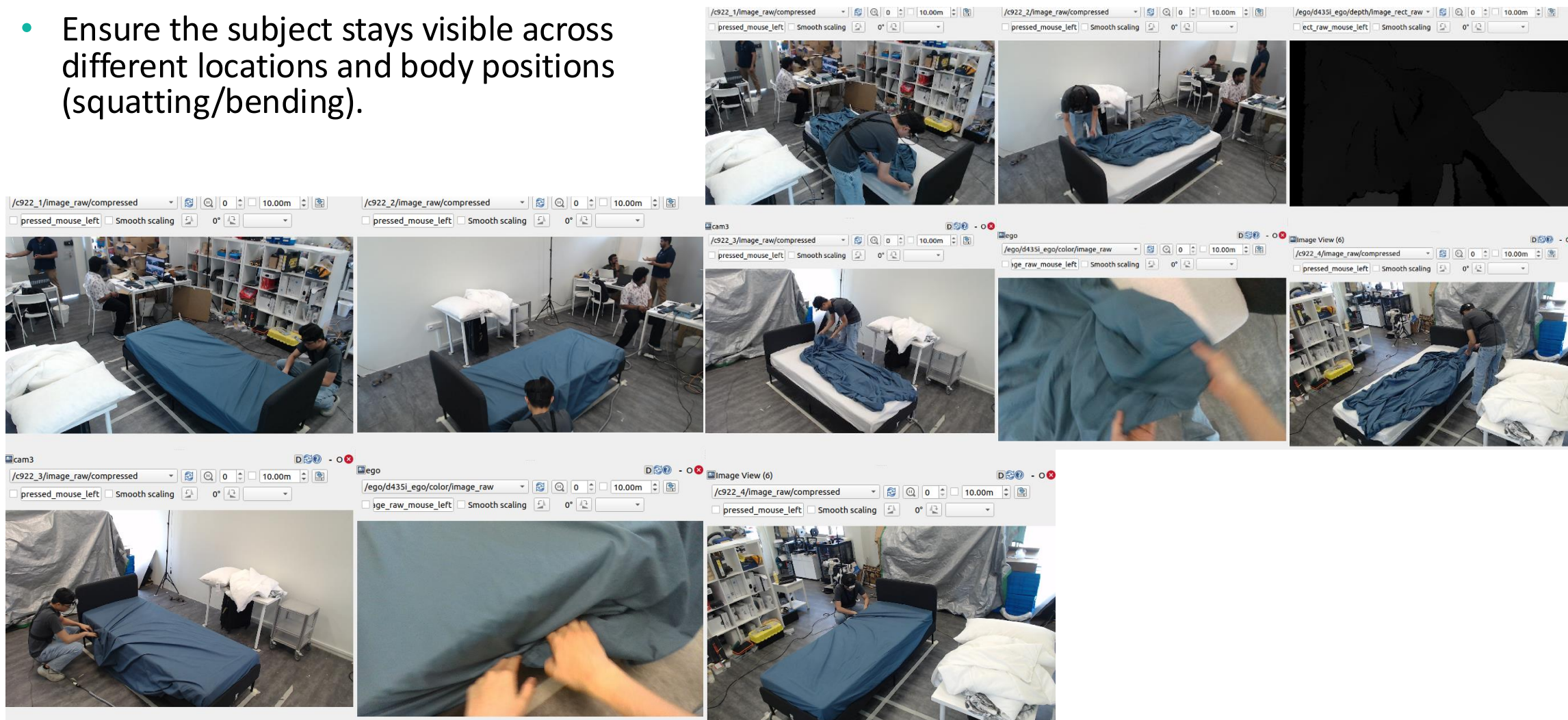
- Exo camera view: the cameras should capture the subject from the knees to the head of the person performing the manipulation task
- Ego camera view: the camera should be centred on the forehead, with rotation adjusted to capture the hands & forearms of the person performing the task.



- Adjust the camera view, orientation, and the bed and table's position to achieve a clear view of the subject.

Camera Setup — Subject Visibility

- Ensure the subject stays visible across different locations and body positions (squatting/bending).



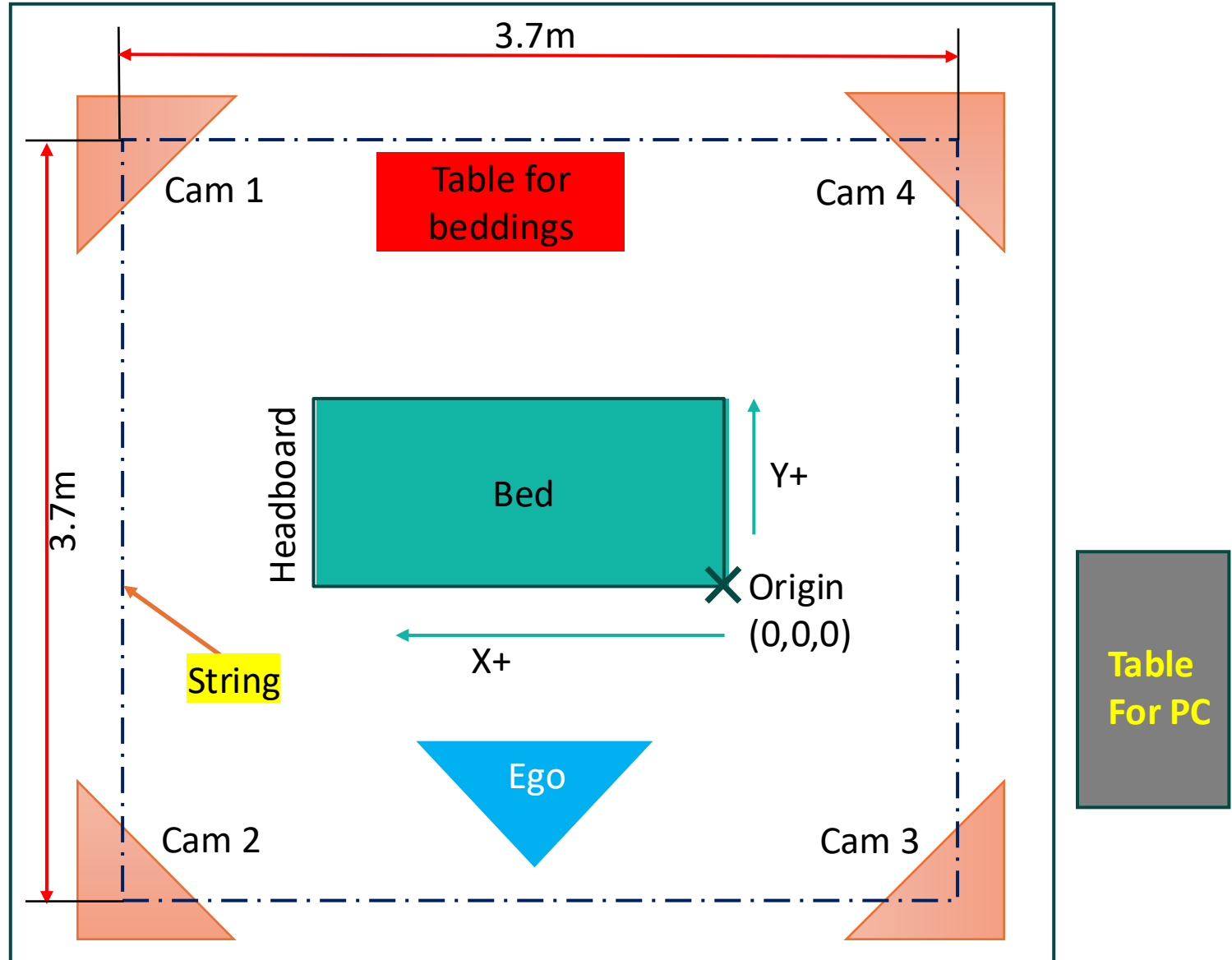
Mark the Setup

- Once the cameras are adjusted and the person manipulating the task is visible in all the 4 exo cams, then the position of the bed is finalized & fixed, and the origin is set as the left end of the bed.
- Adjust the table position until cam 1 and cam 4 get a good view of the table, then finalize and fix it.
- Mark and lock the positions of the bed, table and camera stands with tape — or string if tape is not allowed.



Final Setup

- Place the PC running the ROS node near the rig, leaving enough slack in the ego camera's USB cable for the subject to move freely around the bed.



ROS Node Setup

ROS Stack

Install the following components

Component	Version
Ubuntu	22.04.5 LTS (jammy)
Kernel	6.8.0-x-generic (HWE)
ROS 2	Humble
librealsense2	>2.57.7 (apt)

Librealsense2 v2.57 is the minimum required

Kernel 6.8 support starts at librealsense 2.57.

Realsense Camera Firmware pinned

Tested on 5.17.3.10

Verify what is actually installed and what the wrapper is linked against:

```
dpkg -l | grep -E "librealsense2|realsense2-camera "  
ldd /opt/ros/humble/lib/librealsense2_camera.so | grep realsense  
# must resolve to librealsense2.so.2.57 - .so.2.56 means the upgrade did not take
```

Plug in the cameras

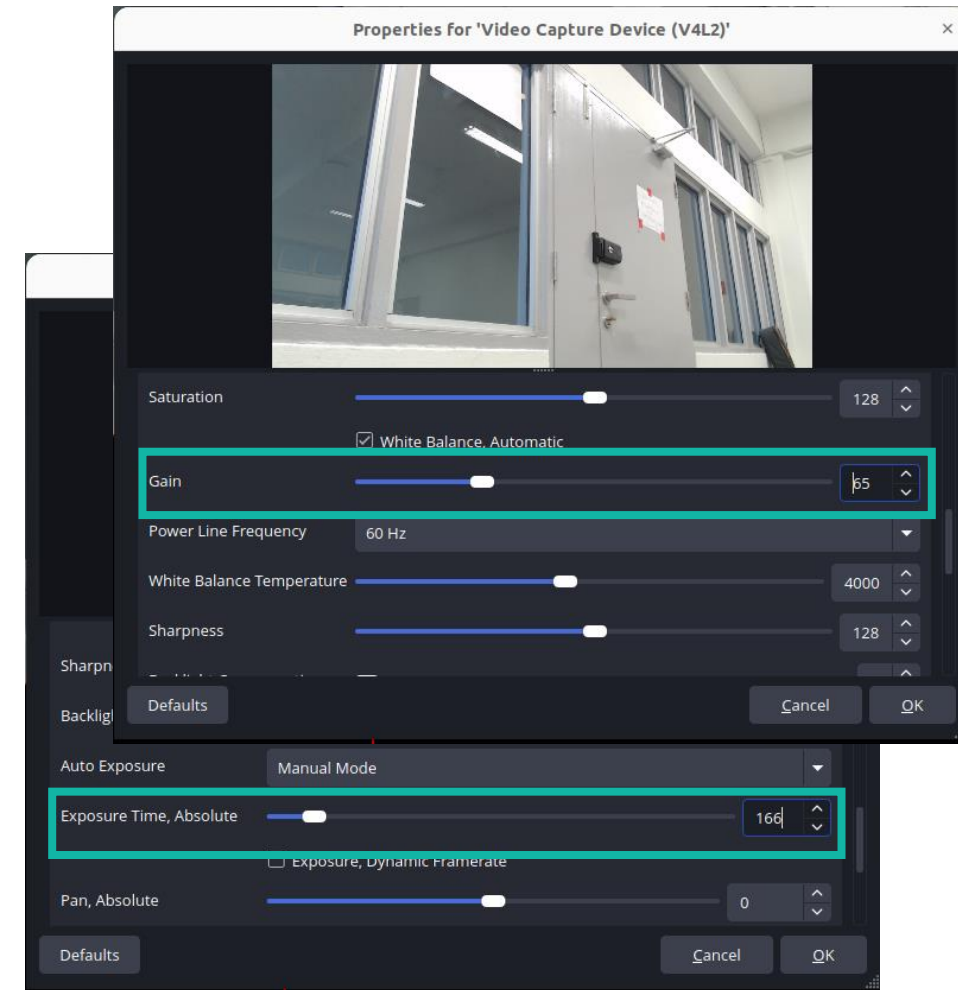
- **Ego D435i → a blue / SS USB-3 port**
 - Use USB 3.0 Active extension cables for a stable signal over distances > 2m.
- **The exo C922 webcams → any USB-2 port.**
 - Use USB 2.0 active extension cables for a stable signal over distances > 2m
- **If the machine allows it, keep the ego on a different USB controller from the webcams** so it isn't fighting them for bandwidth.

Confirm all four cameras are seen:

```
lsusb # expect 4 cameras
```

Lighting Configuration

- Only when the lighting or layout has changed since last time.
- Open each webcam in OBS → Camera Controls. The preview is exactly what the recorder will capture.
- Set exposure or gain by adding the numbers into the launch file and rebuild using `./rebuild.sh`
 - c922 launch files
 - Realsense launch files config is left alone
- **Auto focus is disabled.** The focus is fixed at the longest focal length by default



Lighting Configuration

- C922 launch file edits.
 - Under DEFAULTS object, edit the exposure and gain

```
DEFAULTS = {  
  'width': 1280,  
  'height': 720,  
  'fps': 30.0,  
  'autoexposure': False,  
  # Shutter speed, in 100us units: value = 10000 / denominator  
  # 1/30 s -> 333 1/125 s -> 80  
  # 1/60 s -> 167 1/250 s -> 40  
  # Range 3 (~1/3333 s) .. 2047 (~1/4.9 s). Ignored while autoexposure.  
  'exposure': 166,  
  # The ISO analogue. There is no ISO control -- this is raw sensor gain,  
  # 0 (cleanest) .. 255 (brightest, noisiest), uncalibrated. Raise it only  
  # when a short enough shutter leaves the image too dark.  
  'gain': 25,  
  'autofocus': False,  
  'focus': 0, # 0 (far) .. 250 (near), steps of 5.  
}
```

Lighting Configuration

- Minimum Lighting: no shadows on the person or the bedding. (Lux meter: 250 -300 lux)
- Adjust camera exposure to keep the subject/beddings patterns visible.
- Subject/beddings should be distinct from the background
- Avoid directional lighting. Prefer fill lighting.



Too Dark: unable to separate the subject/beddings from the background



Too bright: unable to distinguish bedding features or the subject



Right Exposure: subject/bedding are clearly visible and identifiable.

Camera Serials

- Both drivers find cameras by **hardware serial**, which is constant across replugs.
- Serials are baked into the launch files. A new or swapped camera does not appear until its serial is added and the workspace rebuilt.

```
rs-enumerate-devices | grep "Serial Number" # realsense
ls /dev/v4l/by-id/ | grep -i c922 # logitech
# unsure which physical camera is which? unplug one and diff
the list
```

Two traps that cost a whole session

Use Serial Number, never Asic Serial Number.

The by-id path contains the ASIC serial. Wrong number and the camera enumerates fine but never starts.

Always -video-index0.

Index 1 is a metadata node and will never stream.

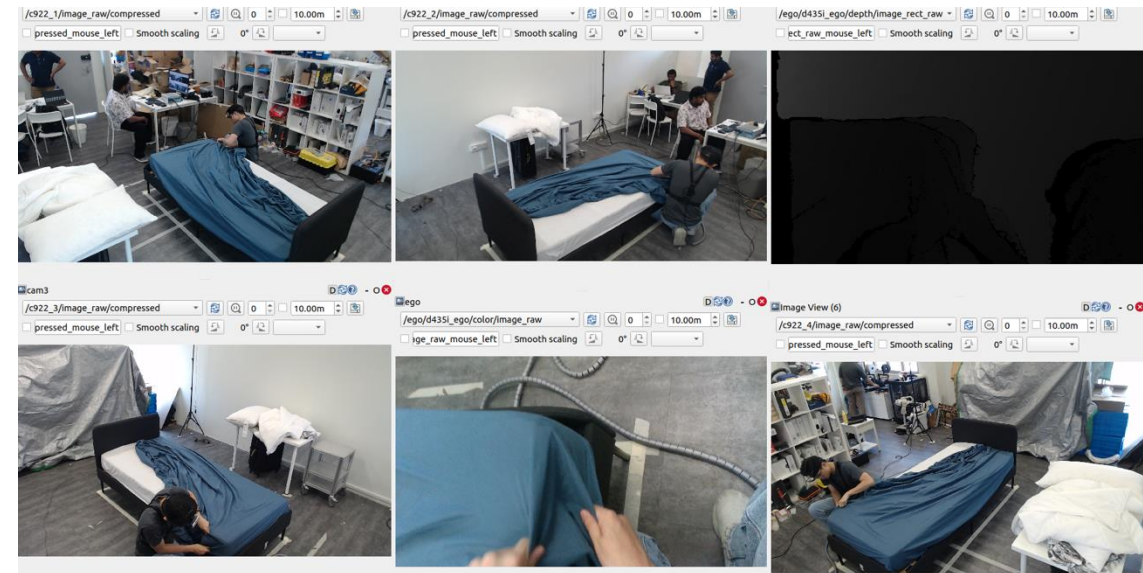
Then rebuild. The serial and the C922 device path each appear in their own c922 and d435i launch files, so the node and its preflight check cannot drift apart when a camera is swapped.

Start the cameras (setup)

- Terminal 1 - for the cameras
- `./rebuild.sh`
- `./cameras.sh` – in the same terminal
 - Ctrl-C here stops the cameras
- **The first few seconds look wrong.** Exposure, gain and focus apply a moment after each stream opens
- Terminal 2: `rqt`
 - One view per camera.

Measure a feed that looks laggy:

```
ros2 topic hz /c922_1/image_raw/compressed
# expect ~30
```



Common faults — Startup

IT REFUSES TO START — any one check kills the whole launch

Message	What to do
no install/setup.bash - not built	Run ./rebuild.sh.
D435i not detected on USB	Plug the ego camera in.
D435i negotiated 480 Mbps, needs 5000+	Wrong port or cable - blue/SS + stock cable.
D435i depth units 0.002, expected 0.001	Replug the D435i - it resets to 0.001.
C922 not found - unplugged or serial moved	Replug the c922 and restart camera.sh
C922 already open	Close it - usually OBS.

Came up, but not healthy

Depth stream start failure - Hardware Error

Replug the D435i, directly to the PC if possible instead of over a USB hub.

Frames didn't arrive within 5 seconds

Bandwidth - move the D435i to another controller.

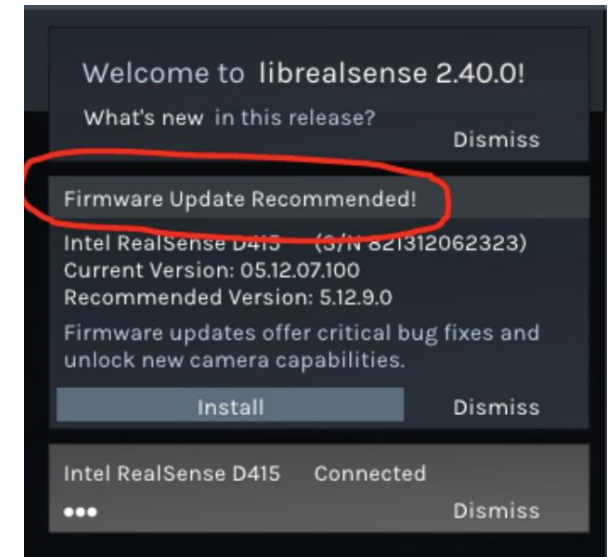
Calibrate the Rig

Calibration

- Ego Camera Calibration – this calibration is stored on the Realsense camera itself. It should not need recalibrating regularly.
- Exo camera calibration
 - Calibration is only good until there is a change to the:
 - Bed position
 - Camera rotation or position.
 - Any change that cannot be returned to the calibrated state (tipping over of the camera stand), requires the recalibration of the setup.
 - To help track which calibration file belongs to each recording, place the JSON in the session folder
 - This step is just for housekeeping
 - In extraction the calibration data is added to the metadata.json

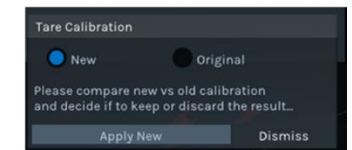
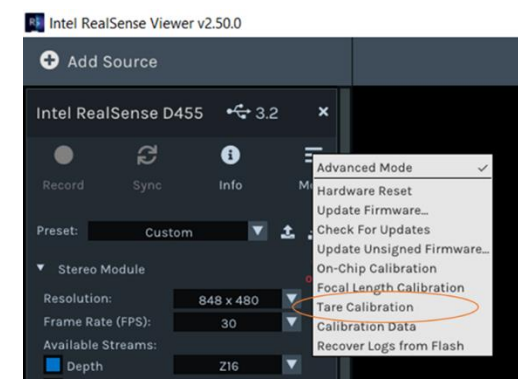
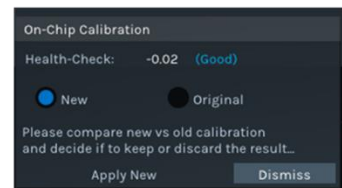
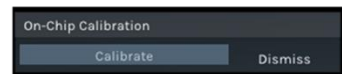
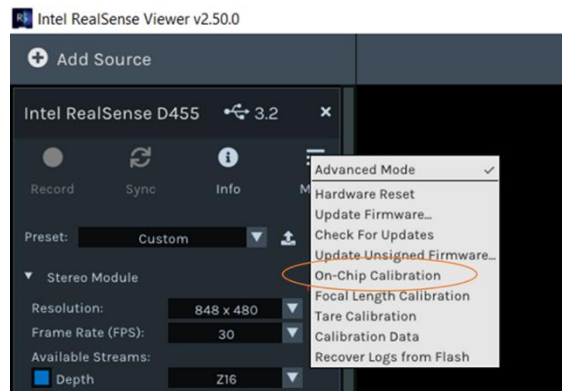
Ego Camera Calibration (D435i/if)

- Firmware Update
 - Use Realsense Viewer to update the firmware. Version tested on 5.17.3.10
 - A popup at the top right-hand corner will prompt you to update.
- IMU Calibration
 - [Guide](#)
 - Follow steps 4.1–4.5, only the components relevant to the D435i.



Ego Camera Calibration (D435i/if)

- [Guide](#)
- Set up the D435i module on a stand, 1.2 metres from a flat wall
- On-Chip Calibration
 - From the Menu button, select On-Chip Calibration, click calibrate, wait for calibration to complete and click Apply New
- Tare Calibration
 - Click Get
 - Click Calibrate, wait for calibration to complete and click Apply New



Ego Camera Calibration (D435i/if)

- Adjust the headband to ensure that hands and objects are placed centrally in frame
- A test run can be done by picking up, carrying and placing down a pillow.



Exo-Camera Calibration — Capture

- Clone the leo-exo-calib repository
- Generate the pdf for the Charuco board. It is generated at A1 size
- Print at 100% scale and mount on a hard board so it's easy to carry around

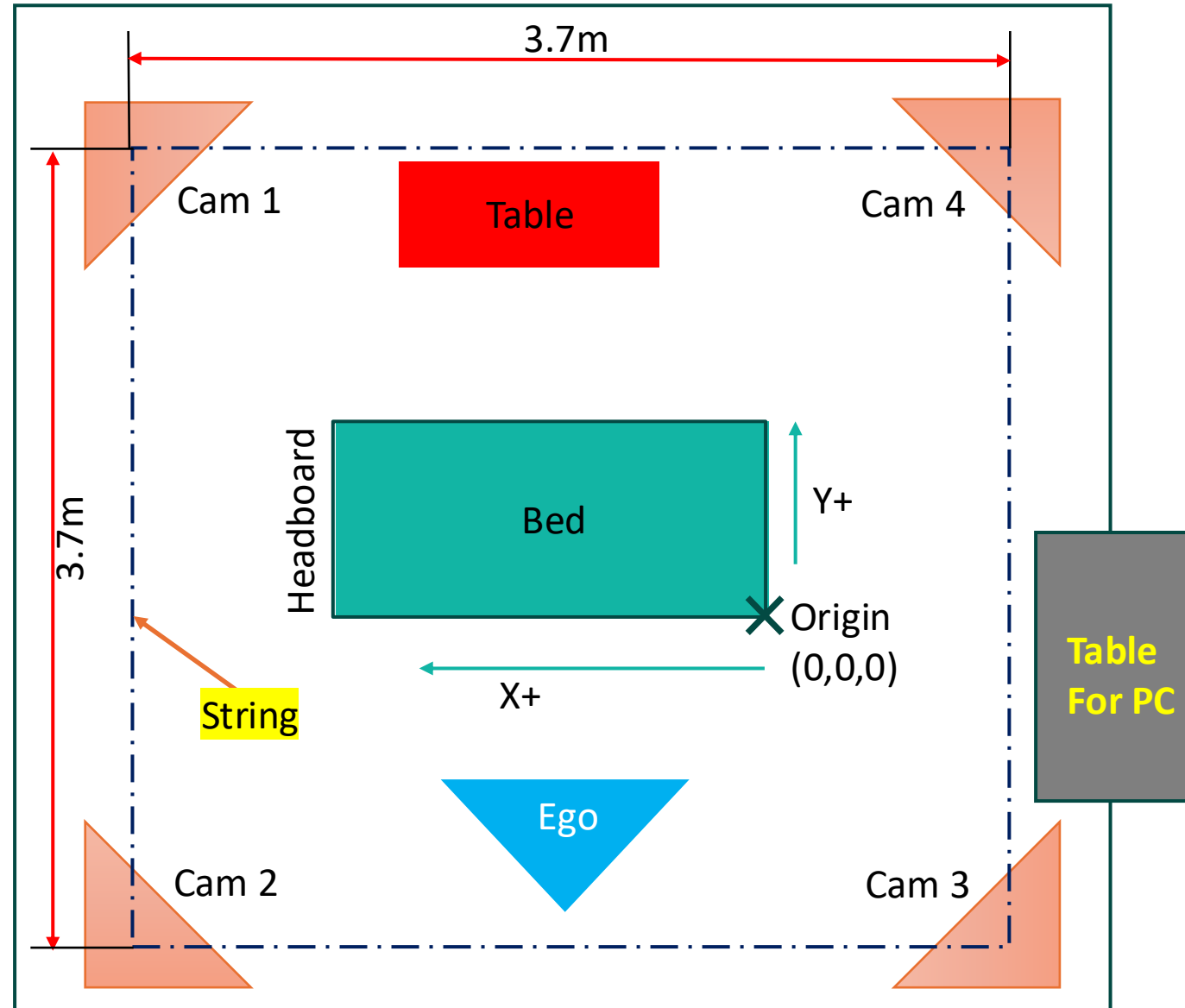
```
// cmd to run to generate pdf and json of the board  
python make_board.py --out board/
```

Exo-Camera Calibration

- Capture

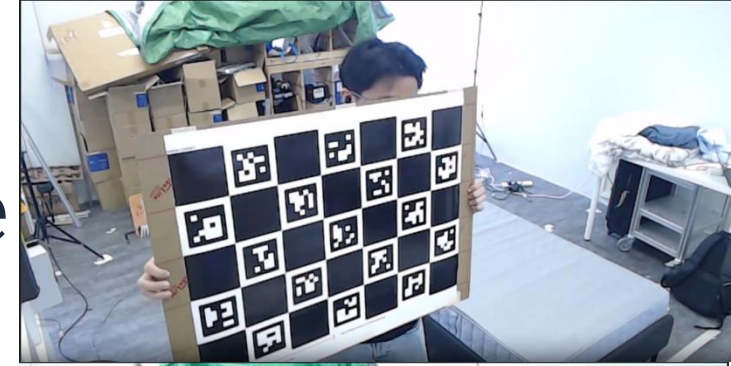
- Camera_placement.json
 - Take the measurements of the 3D vector from origin (floor) to the camera mount, where the camera connects with the stand.
 - X axis is positive towards headboard.
 - Y axis is positive away from origin.
 - Z axis is 0 on the floor, positive moving upwards.

```
// In camera_placement.json
{
  "exo_cam1": [2.67, 2.315, 1.99],
  "exo_cam2": [2.67, -1.44, 1.99],
  "exo_cam3": [-1.2, -1.44, 1.99],
  "exo_cam4": [-1.2, 2.315, 1.99]
}
```



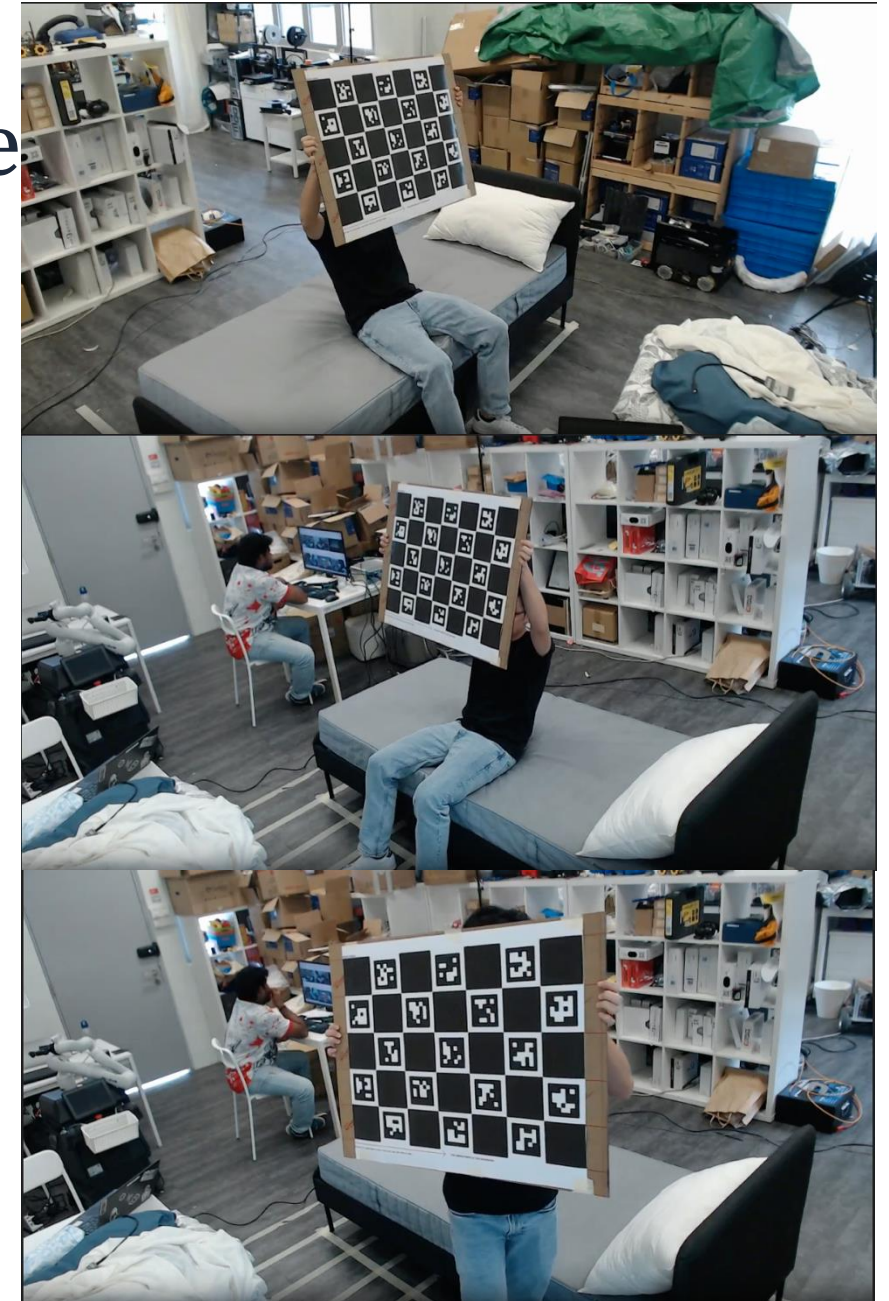
Exo-Camera Calibration — Capture

- Intrinsics
 - Ensure the settings match those used to record your data.
 - Camera position, locked focus (disable autofocus), resolution
 - Record video of the charuco board, moving it across the camera frame. One video per camera.
 - **Name the files as follows: `exo_cam1.mp4` ... `exo_cam4.mp4`**
 - **Can be done with OBS.**
 - Required for usable data
 - Move the board from corner to corner covering full area.
 - Ensure the board is large in the frame. About half the area.
 - Move slowly to prevent blurring
 - Ensure the markers are not blurred or obscured by lighting glare.
 - Recommended for better accuracy
 - Rotate the board about the vertical and horizontal axes.
 - Move the board closer and further away from the camera.
 - Repeat this for each camera.



Exo-Camera Calibration — Capture

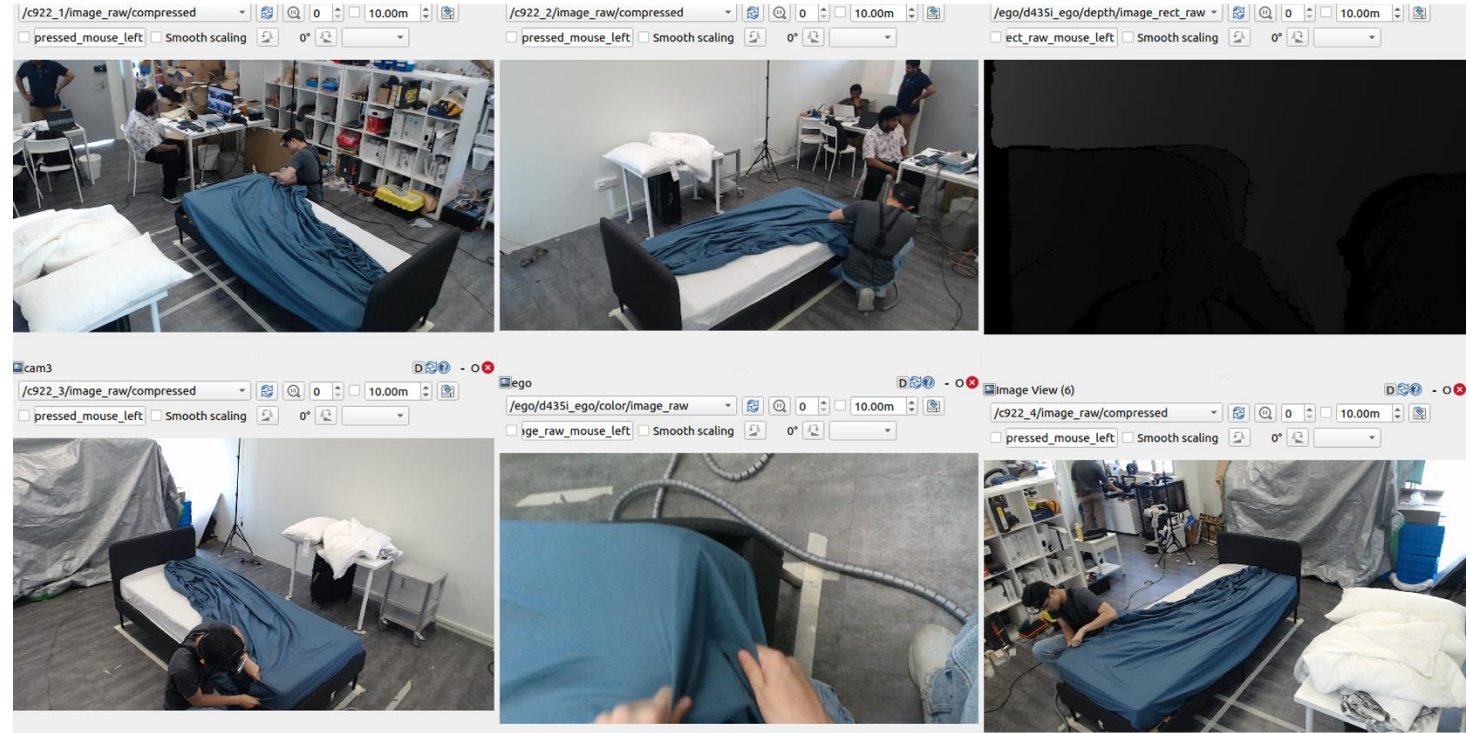
- Extrinsic
 - Ensure the settings match those used to record your data.
 - Camera position, locked focus (disable autofocus), resolution
 - Record video walking between cameras with the charuco board.
 - **This uses the recording setup to capture 4 synchronised videos**
 - Required for usable data
 - Move slowly to prevent blurring
 - Ensure the markers are not blurred or obscured by lighting glare.
 - Ensure that the board at certain points is in view of neighbouring cameras.
 - Ensure the board is viewed from different points in the recording space, and at different angles
 - Recommended for accuracy
 - Capture the board fully in frame



Exo-Camera Calibration ROS Node component

- In leo-data-collection root
- Use the extrinsics ROS node commands. These are used to record a synced video feed of all 4 cameras

```
// launch rqt in first terminal\  
> rqt  
  
// run in second terminal  
// run from leo-data-collection root  
> ./rebuild.sh  
> ./exo_calib_launch.sh
```



Exo-Camera Calibration ROS Node component

```
(base) ✓ ~/leo_ws [master|+ 2]
17:06 $ ./exo_calib_record.sh walk-about1

waiting for all 4 streams (up to 30s)...
all 4 stream(s) live after 8.0s

recording session walk-about1' -> 2026-07-31_walk-about1
8 topics, watching 4 for silence every 5s
Ctrl-C to stop. Do NOT close this window.
SIGHUP leaves the bag unreadable.

[INFO] Subscribed to topic '/c922_1/image_raw/compressed'
[INFO] ... 15 topics subscribed ...
[INFO] Recording...

closing the bag -- do not interrupt
[INFO] Writing remaining messages to the bag...
[INFO] Recording stopped
bag closed after 1s

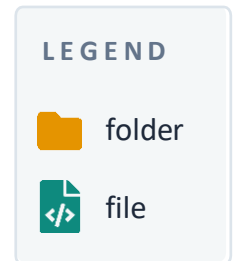
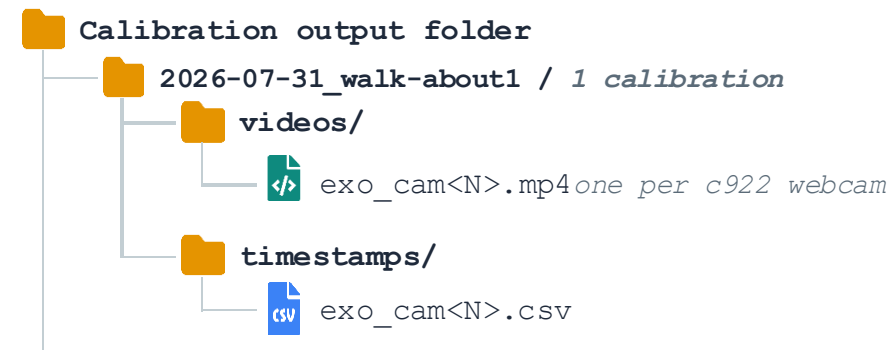
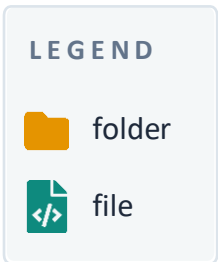
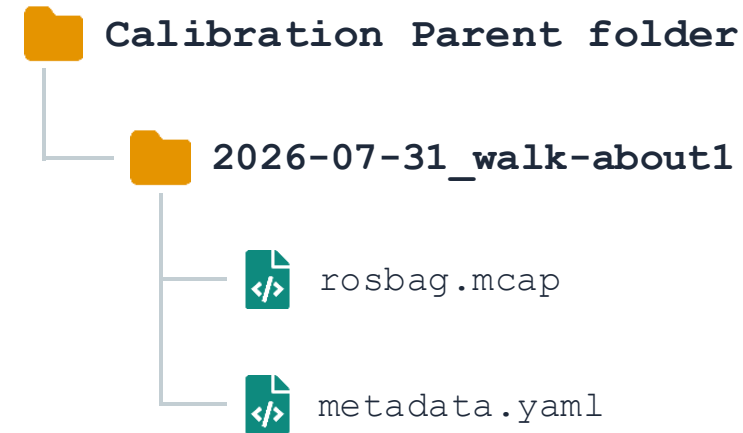
stopped after 14s -- 2026-07-31_walk-about1

Files:      2026-07-31_walk-about1_0.mcap
Bag size:   2.2 GiB
Duration:   12.831976494s
Messages:   4235
Topics:     /c922_1/image_raw/compressed  385
            /c922_2/image_raw/compressed  386
            /c922_3/image_raw/compressed  385
... 8 topics total ...
```

- In leo-data-collection root
- Run `./exo_calib_record.sh <folder name>` to start recording.
 - `./exo_calib_record.sh walk-about1`
- **It watches all 4 exo camera streams for silence every 5s.** Recording ends within 7s of a camera disconnect.
- **Recording ends with a stated cause:** a dead camera or a full disk.
- **Do NOT close the window.** Ctrl-C to stop. Closing it sends SIGHUP and leaves the bag unreadable.

Calibration Extraction

- Output of the extrinsics recording is a ROSbag folder like '2026-07-31_walk-about1' folder
- The extraction script expects multi-episode session, so the calibration parent folder is required.
- The extraction scripts will also expect a metadata JSON file. You can provide the one found in the GitHub repo.
- The extraction script takes the following references
 - Input - Calibration parent folder
 - Output – calibration output folder



Exo-Camera Calibration — Solve

- Once we have the following data
 - Board.json
 - Videos from intrinsics
 - Keep all 4 videos in one folder: each video name follows the camera it records from
 - Intrinsics video folder > exo_cam1.mp4, exo_cam2.mp4, exo_cam3.mp4, exo_cam4.mp4
 - Videos from extrinsics
 - Use the folder that is created by the rosbag extraction script
 - **Read the extraction section for full details of the output folder and the extraction process**
- Run the scripts in order
 - Intrinsics script
 - Extrinsics script
- Output is a single JSON file. This is to be applied to all the recordings using this rig

```
// in the conda env
// in the leo-exo-calib root
// run the intrinsics script
> python exo_intrinsics.py \
  --intrinsics /path/to/calib_intrinsics \
  --board board/board.json \
  --out calib/intrinsics.json

// run the extrinsics script
> python exo_extrinsics.py \
  --walk /path/to/calib_walk \
  --calib calib/intrinsics.json \
  --camera_positions camera_positions.json \
  --board board/board.json \
  --out calib/calib.json

>
```

Exo-Camera Calibration — Solve

- Understanding the output JSON
 - Cameras_solved: the number of cameras calibrated successfully
 - Accuracy
 - Relative headline 'worst_m' the agreement amongst different cameras in metres. Target under 0.05 metres
 - Absolute headline 'rms_m' the root mean square of the predicted camera position in metres. Target under 0.1 metres
 - Intrinsic
 - Fx_spread_pct. The spread of the fx value should be under $\sim 3\%$, indicating the cameras agree, which is expected since they are all the same model.
 - If any of the cameras are not resolved:
 - Retake the intrinsics video and test calibration
 - If it still fails, retake the extrinsics walk-about video as well.

```
{
  "scene_details": {
    "board": { ...board data... } // 7x3 ChArUco, 100mm squares, dict, print scale
  },
  "cameras": { // one entry per camera (exo_cam1..4)
    "exo_cam1": {
      "intrinsics": {
        "K": [[fx,0,cx],[0,fy,cy],[0,0,1]], // camera matrix: focal length + principal point (px)
        "dist": [k1,k2,p1,p2,k3], // OpenCV distortion (radial + tangential)
        "image size": [1280, 720] // frame [width, height], px
      },
      "status": "ok", // per-camera solve result
      "extrinsics": {
        "T_world_cam": [ 4x4 [R | t] ], // pose world<-cam: R rotation, t translation (m)
        "position_m": [x, y, z] // camera position in world frame (m)
      }
    }
    // exo_cam2 ... exo_cam4: same structure
  },
  "accuracy": {
    "relative": { // inter-camera agreement (triangulation precision)
      "worst_m": 0.0, // worst pair, m | sigma (headline, lower better)
      "per_pair_m": { ... } // one value per overlapping camera pair
    },
    "absolute": { // fit to spec-measured positions (catches scale error)
      "rms_m": 0.0, // RMS over cameras, m (headline, lower better)
      "per_camera_m": { ... } // residual per camera to its measured position
    },
    "intrinsics": { // per-camera lens solve quality
      "fx_spread_pct": 0.0, // spread of fx across identical lenses (want <~3%)
      "per_camera": {
        "exo_cam1": {
          "fx": 0.0, // solved focal length, px
          "coverage": 0.0, // frame fraction reached (want >0.45)
          "rms_px": 0.0, // reprojection error, px (sanity, not proof)
          "angle_spread": 0.0, // board tilt variety (low = under-constrained)
          "n_views": 0 // frames used (context only)
        }
      }
    }
  },
  "cameras_solved": "4/4" // cameras that solved successfully
}
```

Recording one session

Recording Overview

- Only one subject per session
 - Ensure only one person is in frame.
- Each person taking part in the dataset should be assigned a unique UUID v4. We can track a user across the different videos they are in.
 - <https://www.uuidgenerator.net/version4>
- One recording session applies for all the episode recordings that cover
 - 1 complete workflow pass of bed making
 - Done by 1 subject
 - For 1 Bed type.
 - E.g one session = 1 workflow pass's episodes done by subject A on a Super Single bed.

Three terminals, started in order

#	Terminal	You run	Stays up for
1	cameras	<code>./cameras.sh</code>	the all sessions
2	monitor	<code>rqt</code>	the all sessions
3	recorder	<code>./record.sh</code> or <code>./oops.sh</code> to mark mistake episode	one episode

Terminals 1 and 2 are set-and-forget

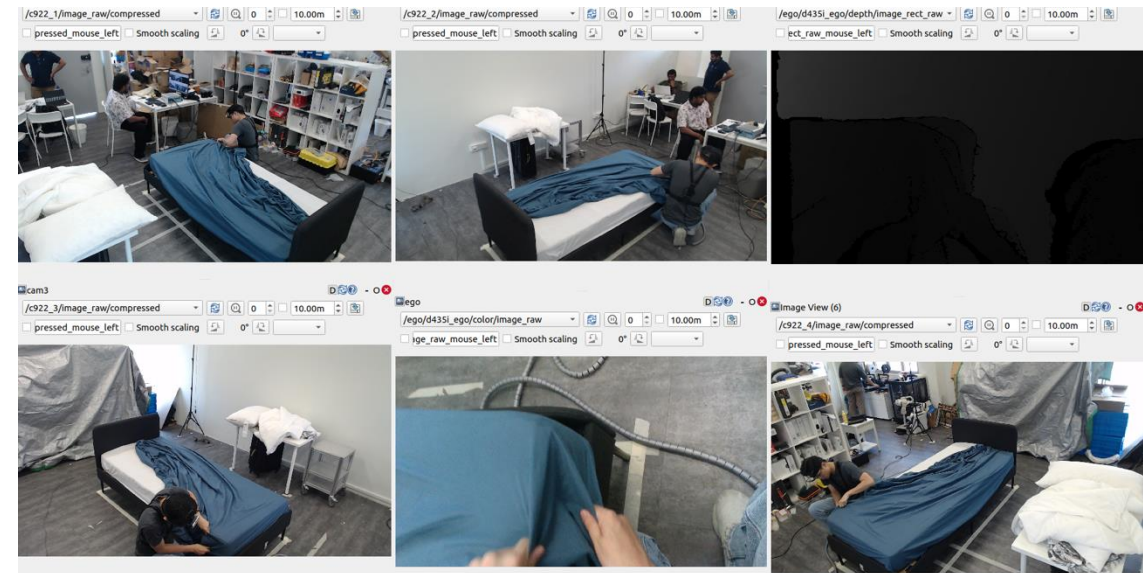
Leave the cameras and the monitor running all day. You only interact with terminal 3 while recording.

Start the cameras (recording day)

- Terminal 1 – in data collection repository
- `./rebuild.sh`
- `./cameras.sh` – in the same terminal
 - Ctrl-C here stops the cameras
- **The first few seconds look wrong.** Exposure, gain and focus apply a moment after each stream opens
- Terminal 2: `rqt`
 - One view per camera. Check every feed is aimed, focused, unobstructed and well-lit

Measure a feed that looks laggy:

```
ros2 topic hz /c922_1/image_raw/compressed
# expect ~30
```



Run Recorder

```
(base) ✓ ~/leo_ws [master|+ 2]
17:06 $ ./record.sh 23

waiting for all 5 streams (up to 30s)...
all 5 stream(s) live after 8.0s

recording session 23' -> 2026-07-31_23_1
15 topics, watching 5 for silence every 5s
Ctrl-C to stop. Do NOT close this window.
SIGHUP leaves the bag unreadable.

[INFO] Subscribed to topic '/c922_1/image_raw/compressed'
[INFO] ... 15 topics subscribed ...
[INFO] Recording...
[INFO] [rosbag2_recorder]: All requested topics are subscribed. Stopping
discovery...

closing the bag -- do not interrupt
[INFO] Writing remaining messages to the bag...
[INFO] Recording stopped
bag closed after 1s

stopped after 14s -- 2026-07-31_23_1

Files:      2026-07-31_23_1_0.mcap
Bag size:   2.2 GiB
Duration:   12.831976494s
Messages:   4235
Topics:     /c922_1/image_raw/compressed  385
            /c922_2/image_raw/compressed  386
            /c922_3/image_raw/compressed  385
            /ego/.../color/image_raw      385
            /ego/.../aligned_depth        384
            /ego/.../depth/image_rect_raw 384
            ... 15 topics total ...
```

- In data collection root
- Run `./record.sh <episode code>` to start recording.
 - `./record.sh 23`
- **Wait for the ‘All request topics are subscribed. Stopping Discovery...’ message** before telling the subject to begin their action.
- **Call out the end of recording to the subject**, so they know when to relax and when to keep still. This reduces non-trainable actions like adjusting glasses or the ego camera.

Run Recorder

```
(base) ✓ ~/leo_ws [master|+ 2]
17:06 $ ./record.sh 23

waiting for all 5 streams (up to 30s)...
all 5 stream(s) live after 8.0s

recording session 23' -> 2026-07-31_23_1
15 topics, watching 5 for silence every 5s
Ctrl-C to stop. Do NOT close this window.
SIGHUP leaves the bag unreadable.

[INFO] Subscribed to topic '/c922_1/image_raw/compressed'
[INFO] ... 15 topics subscribed ...
[INFO] Recording...
[INFO] [roslauncher]: All requested topics are subscribed. Stopping
discovery...

closing the bag -- do not interrupt
[INFO] Writing remaining messages to the bag...
[INFO] Recording stopped
bag closed after 1s

stopped after 14s -- 2026-07-31_23_1

Files:      2026-07-31_23_1_0.mcap
Bag size:   2.2 GiB
Duration:   12.831976494s
Messages:   4235
Topics:     /c922_1/image_raw/compressed  385
            /c922_2/image_raw/compressed  386
            /c922_3/image_raw/compressed  385
            /ego/.../color/image_raw      385
            /ego/.../aligned_depth        384
            /ego/.../depth/image_rect_raw 384
            ... 15 topics total ...
```

- **It watches all five camera streams for silence every 5s.** Recording ends within 7s of a camera disconnect.
- **Recording ends with a stated cause:** a dead camera or a full disk.
- **Do NOT close the window.** Ctrl-C to stop. Closing it sends SIGHUP and leaves the bag unreadable.

Stop safely, then verify

- **Stop recorder.** Ctrl-C terminal 3 (record.sh) and let it finish - it writes the bag's index at close.

Check the bag if needed:

```
ros2 bag info <session-name>
```

A good bag has

The right duration.

Matches what you recorded.

~30 messages/sec per camera.

Far fewer means dropped frames - the disk couldn't keep up.

The episode track is present,

if you marked episodes. A missing track fails silently and is costly to fix later.

```
(base) ~/leo_ws [master|+ 2]
17:23 $ ./record.sh 1

=== BAG SUMMARY ===
Files           : 2026-08-07_test_1_0.mcap
Bag size        : 406.3 MiB
Storage id      : mcap
Duration        : 3.091958871s
Start           : Aug  7 2026 17:23:12.854830038 (1786094592.854830038)
End             : Aug  7 2026 17:23:15.946780909 (1786094595.946780909)
Messages        : 1637

=== TOPIC INFORMATION ===
#  Topic                                           Type                                           Count  Serialization
-  -
1  /ego/d435i_ego/imu                             sensor_msgs/msg/Imu                         556    cdr
2  /ego/d435i_ego/extrinsics/depth_to_gyro        relatense2_camera_msgs/msg/Extrinsics        1      cdr
3  /ego/d435i_ego/color/camera_info               sensor_msgs/msg/CameraInfo                   84     cdr
4  /c922_1/camera_info                             sensor_msgs/msg/CameraInfo                   93     cdr
5  /c922_1/image_raw/compressed                   sensor_msgs/msg/CompressedImage              93     cdr
6  /c922_2/camera_info                             sensor_msgs/msg/CameraInfo                   93     cdr
7  /c922_2/image_raw/compressed                   sensor_msgs/msg/CompressedImage              93     cdr
8  /c922_3/camera_info                             sensor_msgs/msg/CameraInfo                   93     cdr
9  /c922_3/image_raw/compressed                   sensor_msgs/msg/CompressedImage              93     cdr
10 /c922_4/camera_info                             sensor_msgs/msg/CameraInfo                   93     cdr
11 /c922_4/image_raw/compressed                   sensor_msgs/msg/CompressedImage              93     cdr
12 /ego/d435i_ego/depth/image_rect_raw            sensor_msgs/msg/Image                        83     cdr
13 /ego/d435i_ego/depth/camera_info               sensor_msgs/msg/CameraInfo                   83     cdr
14 /ego/d435i_ego/color/image_raw                 sensor_msgs/msg/Image                        84     cdr

=== RECORDING INFO ===
Recording started at : Aug  7 2026 17:23:12
Recording ended at   : Aug  7 2026 17:23:15
Total duration       : 3.09 seconds
Total messages       : 1637
Average rate         : 530.18 messages/second

(base) ~/leo_ws [master|+ 2]
17:23 $
```

Stop safely, then verify

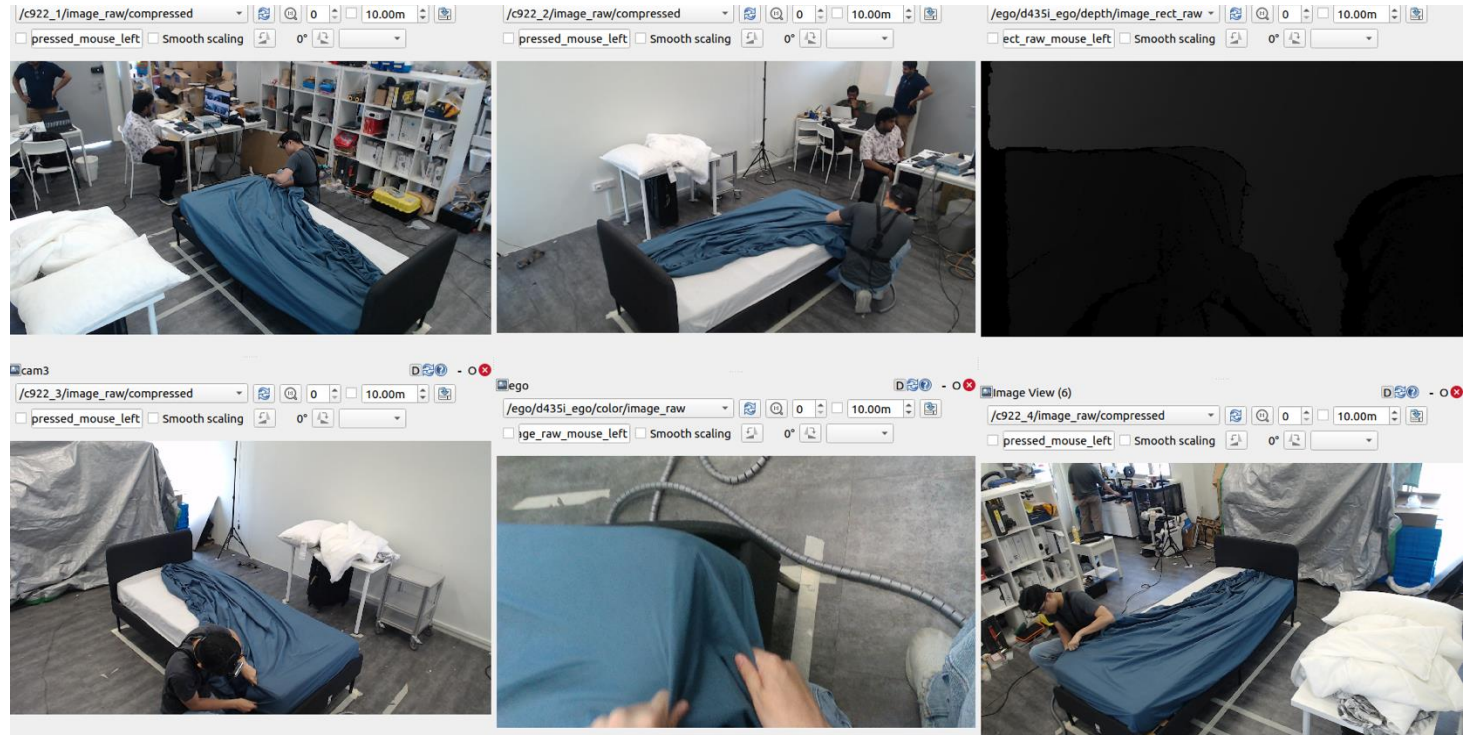
- **Visually verify if needed.**

Open the RQT window from a separate terminal

```
rqt
```

Play back the bag

```
ros2 bag play <session-name>
```



Episode Mistakes annotation

```
(base) ✓ ~/leo_ws [master|+ 1...1 1]
17:21 $ ./oops.sh 2026-08-07_test-imu_1 m1
    marked 2026-08-07_test-imu_1: m1
(base) ✓ ~/leo_ws [master|+ 1...1 1]
17:21 $ ./oops.sh 2026-08-07_test-imu_1
    usage: ./oops.sh <episode-folder> <code> [<code> ...]
(base) ✗-1 ~/leo_ws [master|+ 1...1 1]
17:21 $
```

- An episode experiences one of the predefined categories of mistakes (see earlier slide)
 - If a mistake is recognised mid-episode, continue and fix the action within the same episode recording. Label that whole episode as a mistake
 - If the mistake is noticed after the episode is completed, label the completed episode as a mistake, then run a repeat episode to fix it.
- oops.sh uses predefined mistake codes (m1, m2, ...) only
- ./oops.sh <episode-folder> <code (lowercase)>
Eg: ./oops.sh 2026-08-07_test-imu_1 m1

Common faults — During Recording

DURING A RECORDING — what fails and how to recover

Symptom	What to do
Disk under 1 GB free	record.sh won't start — clear space first.
Disk fills mid-episode	Recording ends — free space, then redo the episode.
A topic goes silent mid-episode	Node halts it loudly (5 s check); data is partial — redo.
Recorder dies mid-episode	Check the bag for usable data; usually redo.
No metadata.yaml after episode	Run <code>ros2 bag reindex -s mcap <dir></code> ; check bag, redo if unsure.

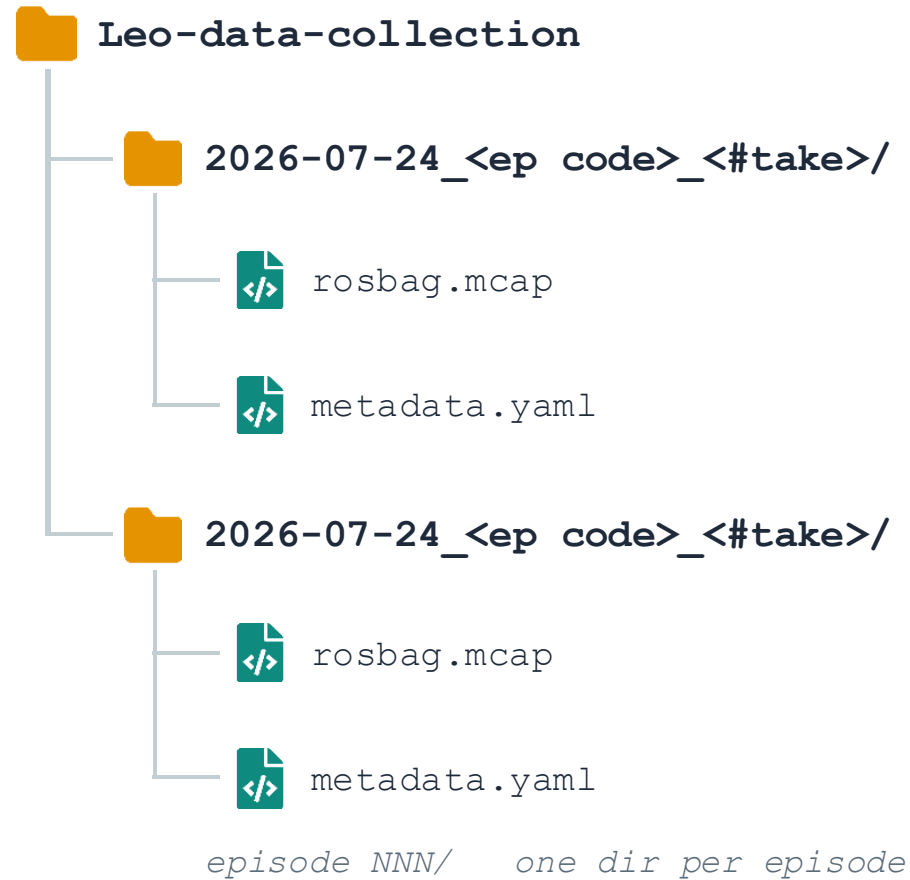
When in doubt, redo the episode

Most failures stop loudly — the recorder checks every topic every 5 s and halts if one goes quiet.

A clean re-take beats debugging a partial bag.

Recording output files

- The recording terminal was being called from leo-data-collection folder
- The Rosbag folders are written to disk in leo-data-collection folder
- The rosbag folder names can be safely renamed, but the files (mcap) cannot as they are referenced by metadata.yaml



LEGEND

 folder

 file

Recording output files

- Put all episodes from one session into a single folder
- The session folder label should include the ISO date (yyyy-mm-dd), UUID of the subject, and the session number.



Extraction

Extraction Scripts

- Default Meta to write to all episodes
 - The values you need to specify are:
 - Environment: institution/building name
 - Subject: uuid v4 to identify the subject throughout the dataset
 - Bedtype: single, queen, king
 - The other values are hardcoded and do not need changing
- Default calib json
 - This is the file path to the output of the calibration process.
 - The calibration values are merged into each episode's metadata file

```
DEFAULT_META = {  
    "dataset_name": "leo",  
    "dataset_version": "4.0",  
    "robot_model": "human",  
    "environment": "sutd parcel d",  
    "setup": "v7",  
    "subject": "",  
    "bed_type": "single",  
}  
DEFAULT_CALIB_JSON = "filepath.json"
```

Extraction Scripts

- Default Cameras
 - This defines the topics to be extracted.
 - Key items that can be changed:
 - Presence of colour ego
 - Presence and number of colour exo
 - Presence of imu or depth

```
DEFAULT_CAMERAS = {  
  "ego": {"present": True,  
    "suffix": "d435i_ego/color/image_raw/compressed",  
    "info_suffix": "d435i_ego/color/camera_info",  
    "compressed": True,  
    "label": "cam_ego",  
    "singleton": True},  
  "exo": {"present": True,  
    "suffix": "image_raw/compressed",  
    "compressed": True, "label": "exo_cam", "ids": [1, 2, 3, 4]},  
  "depth": {"present": True, "suffix": "d435i_ego/depth/image_rect_raw"},  
  "imu": {"present": True, "suffix": "d435i_ego/imu"},  
}
```

Extraction Scripts

- Source data tuple
 1. Path to session directory
 2. Path to output directory
 3. Metadata object override. Set as None to use default
 4. Camera object override. Set as None to use default
 5. Calibration override. Set as None to use default

```
ls = [  
    ("source-session-dir", "output-dir", meta_override, cameras_override, calib_override),  
    ("source-session-dir", "output-dir", None, None, None),  
]
```

Extraction Scripts

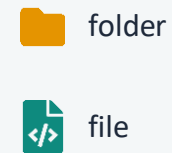
- Wrapper script flow
 - We define the default, and per session overrides.
 - The script runs down the list, processing each item
 - **Outer loop: one session per iteration**
 - **Inner loop: one episode (bag) per iteration inside that session**
 - Per episode — extract the colour, depth and IMU streams, then write the files and metadata.json to output_root/<episode>
 - Result:
 - Every episode folder lands in its respective session folder
 - A summary csv for that session folder.

```
ls = [  
  ("source-session-dir", "output-dir", meta_override, cameras_override, calib_override),  
  ("source-session-dir", "output-dir", None, None, None),  
]
```

Extraction output



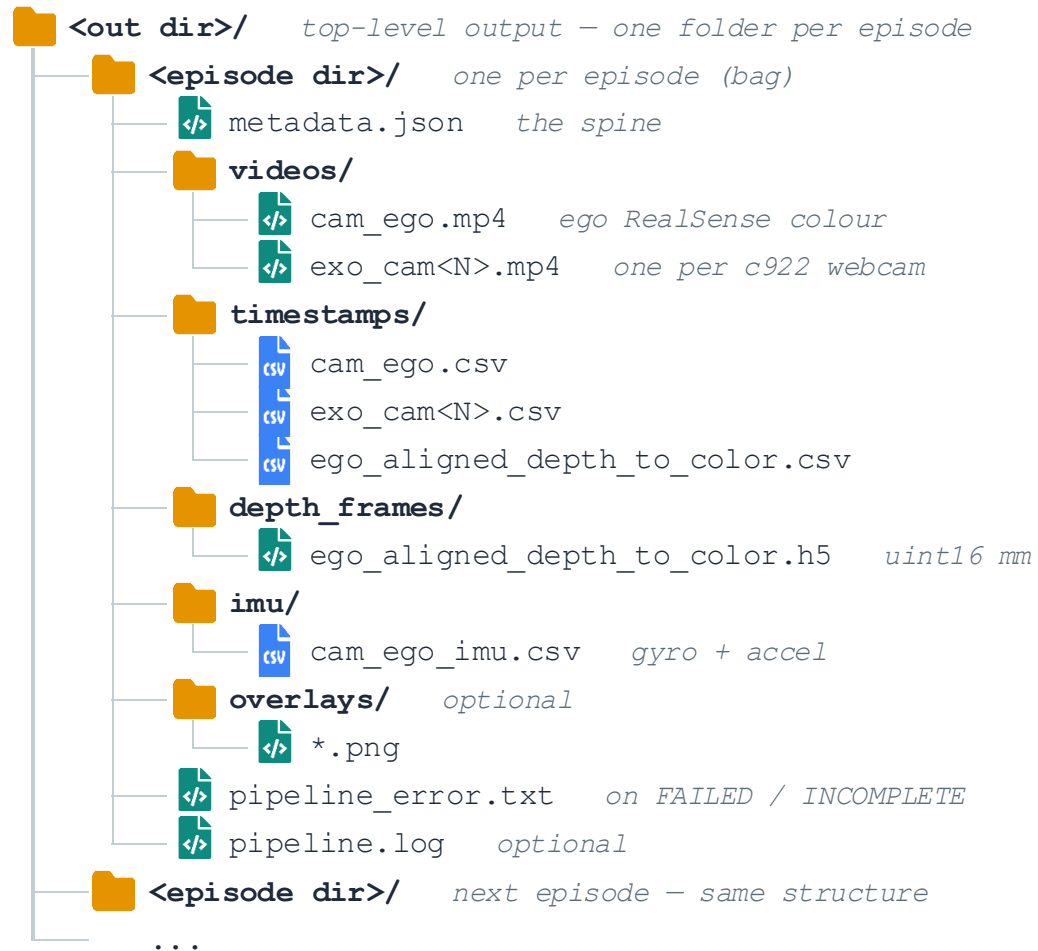
LEGEND



Optional outputs

- Pipeline.log: this is to dump the command line output into a file instead of the command line.
 - Useful for running a long extraction unattended – saves logs for review.
- In wrapper.py: QUIET = True will redirect the output to the log file

Extraction output



Optional outputs

- Overlays: they output the depth frames to a png file for inspection
 - Can only be run by calling `rosbag_process_depth_v3.py -- bag <path/to/bag.mcap> -- overlay-png <num_depth_frames_show>`
- Only useful for debugging, not for use in the dataset.

Metadata.json

```
{
  "metadata": { dataset_name, robot_model, environment, date_recorded,
                setup, subject, bed_type },           // descriptive
  "episode_details": { episode_code, take, start_time_ns, duration_s,
                      mistakes: [ ... ] },           // identity + timing
  "camera_intrinsics": [ { camera, color: {...}, depth: {...} } ], // list, per camera
  "camera_extrinsics": [ { name: "depth_to_color", ... },           // list, per transform
                        { name: "depth_to_gyro", ... } ],
  "steps": {
    "streams": [ { camera, kind, ..., data_error?, timestamps_error? } ],
    "timestamp_range": [ start_s, end_s ],
    "expected_streams": [ "color", "depth", "imu" ], // declared categories (step 1) -
                                                         // legacy category-level fallback; the wrapper path
                                                         // diffs the full cameras config instead
    "missing_stream_error": [ ... ], // presence detail: a declared stream absent from the
                                                         // output - written by the VALIDATOR's diff (legacy
                                                         // metadata may carry extractor-written entries)
    "extra_stream_error": [ ... ], // presence detail: a surplus extracted stream (validator)
    "step_errors": [ ... ], // labels of steps that crashed (isolated loop); paired with
                             // the pipeline_step_error termination token + a traceback in
                             // pipeline_error.txt
    "exo_calib_notes": [ "exo_cam2: status='low_coverage'; K/D and pose set null" ],
                             // ^ one string per broken exo-cam solve (status != "ok");
                             // the status text comes from the calib file. Informational
                             // only - nulls the cam's K/D + pose, never fails the bag.
    "bag_corruption_error": [ ... ] // only when the bag is structurally corrupt
  },
  "termination": { "is_successful": bool, "reason": [ tokens ] } // the verdict
}
```

Session Summary CSV

- The summary includes the metadata identifying the episode details.
 - Completed indicates whether the extraction for that episode completed
 - Exo_calib
 - Full: all 4 cameras calibrated
 - Partial: only 1-3 cameras calibrated
 - None_usable: 0 cameras calibrated
 - Null: no exo cameras found

index	date	episode_code	take	duration_s	mistakes	completed	exo_calib
1	17/8/26	7	1	12.3	[]	TRUE	full
2	17/8/26	7	2	11.8	['m3']	TRUE	partial
3	17/8/26	9	1	13.1	[]	FALSE	none_usable
4	17/8/26	9	2	12.7	[]	TRUE	null

Session Summary CSV

- The summary includes the presence of different types of errors
 - Color
 - Depth
 - IMU

index	color_error	color_timestamp_error	depth_error	depth_timestamp_error	imu_error	imu_timestamp_error	missing_stream_error	extra_stream_error
1	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
2	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
3	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE
4	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE

Session Summary CSV

- The summary includes
 - Output directory path
 - The start ROS time in ns, used to show the sequential order of the episodes in the session.

index	out_dir	start_time_ns
1	out/2026-08-17_7_1	1.7869E+18
2	out/2026-08-17_7_2	1.7869E+18
3	out/2026-08-17_9_1	1.7869E+18
4	out/2026-08-17_9_2	1.7869E+18

The workflow, end to end

1

Set up

Rig, cameras and calibration — once per machine.

2

Record

One subject, one workflow pass per session.

3

Verify

Check each bag: duration, topics, frame rate.

4

Extract

Wrapper script writes one folder per episode.

5

Submit

Output directory holds every session, ready to share.

One subject, one pass, one clean dataset.